

Procédure de déploiement

futurAMI — Assistance Missions Interprètes

Planet Traduction France — Entité Dolibarr 2

Version 1.0 · Août 2026

Front React 18 + Vite · API PHP 8 · MySQL 8 (schéma Dolibarr étendu)

1. Vue d'ensemble

futurAMI est une application web multi-entités qui pilote les missions d'interprétariat, les devis et les factures pour **Planet Traduction France**. L'application s'appuie sur une base MySQL Dolibarr partagée : chaque déploiement est cloisonné par la valeur `entity` (2 pour Planet Traduction France, 1 pour AMI-PTF).

Composant	Technologie
Front-end SPA	React 18, TypeScript, Vite 5, Tailwind CSS, shadcn/ui, motion/react
Back-end API	PHP 8.0+, PDO MySQL, JWT HS256 (auth_helpers.php)
Base de données	MySQL 8 (dbs13098267) — préfixe <code>llx_</code> Dolibarr + tables custom <code>tbl_</code>
Génération PDF	mPDF 8.2 (via Composer)
Serveur web	Apache 2.4+ avec <code>mod_rewrite</code> et <code>mod_env</code>

2. Prérequis serveur

- PHP $\geq$ **8.0** avec extensions `pdo_mysql`, `mbstring`, `gd`, `zip`, `openssl`.
- Serveur MySQL $\geq$ **8.0** accessible ; schéma Dolibarr déjà présent (base `dbs13098267`).
- Apache $\geq$ **2.4** avec `mod_rewrite` et `mod_env` chargés.
- `AllowOverride All` (ou au minimum `FileInfo Options=None Env`) sur le vhost qui sert `/api`.
- Composer installé pour récupérer **mPDF** (utilisé pour les factures PDF).
- Node.js $\geq$ **18** et npm sur la machine de build (jamais installés en production).

3. Build du front-end

Le front React est compilé **avant** déploiement sur la machine de build. Le résultat statique (`dist/`) est ensuite copié tel quel sur le serveur.

```
# Depuis la racine du dépôt futurAMI
npm install                # installe les dépendances (une fois par machine)
npm run build              # produit dist/ (Vite build)

# Vérification rapide : dist/index.html doit exister
ls dist/
```

Info Le proxy Vite `/api` → `http://localhost/futurAMI/api` ne concerne que le dev. En production, le front et l'API doivent être servis **par le même domaine** (ou l'API doit renvoyer les bons en-têtes CORS pour un domaine distinct).

4. Déploiement de l'API PHP

1. Copier le dossier `api/` à la racine web (ex : `/var/www/futurami/api/`).
2. Depuis `api/`, exécuter `composer install --no-dev --optimize-autoloader` pour installer mPDF dans `api/vendor/`.
3. Copier le contenu de `dist/` à la racine publique (ex : `/var/www/futurami/`).
4. S'assurer que le serveur web pointe vers cette racine publique (DocumentRoot).

5. Configuration `.htaccess` de production

Le fichier `api/.htaccess` **n'est pas versionné** (il contient le secret JWT). Il doit être créé **manuellement** sur le serveur avec ce contenu :

```
<IfModule mod_env.c>
# Entité Dolibarr forcée pour ce déploiement
SetEnv DOLIBARR_ENTITY 2

# Secret JWT HS256 – GÉNÉRER UN NOUVEAU ≥ 32 caractères hex pour la prod
SetEnv AUTH_SECRET "REPLACER_PAR_UN_SECRET_UNIQUE_HEX_64_CHARS"

# Durée de vie des tokens en secondes (12 h par défaut)
SetEnv AUTH_TOKEN_TTL 43200

# Mot de passe MySQL production
SetEnv DB_PASS_PROD "REPLACER_PAR_MOT_DE_PASSE_MYSQL_PROD"
</IfModule>
```

Attention sécurité Ne **JAMAIS** réutiliser le `AUTH_SECRET` d'un autre déploiement (notamment AMI-PTF). Utiliser par exemple `openssl rand -hex 32` pour générer un secret dédié à ce déploiement.

Rappel Sans `AUTH_SECRET` valide (≥ 32 caractères), `login.php` répond immédiatement `500 auth_secret_missing`.

6. Base de données

La connexion MySQL est ouverte dans `api/config.php`. Trois éléments sont à vérifier avant la première connexion :

1. **Nom d'hôte, base, user** : édités en dur dans `config.php` (adapter à l'hébergement).
2. **Mot de passe** : lu depuis la variable d'environnement `DB_PASS_PROD` (déclarée dans `.htaccess`), avec fallback en dur — recommandation forte de configurer la variable d'environnement pour ne pas dépendre du fallback.
3. **Entité** : `SetEnv DOLIBARR_ENTITY 2` dans `.htaccess`. Sans cette variable, les super-admins (entity=0) tombent sur le fallback `getConfiguredEntity() = 2`, ce qui est le bon comportement

pour PTF.

Scripts de migration — ordre de lancement

Il n'existe qu'un **seul script SQL manuel à jouer** en production. Tous les autres **ALTER/CREATE TABLE** sont exécutés automatiquement par le code PHP à la volée. Ordre strict à respecter :

#	Étape	Cible	Type d'action
1	Backup complet de la base	MySQL prod	<code>mysqldump</code> avant toute modification.
2	Migration SQL manuelle (unique)	<code>api/requete_sql_entity1_backfill.sql</code>	À jouer via phpMyAdmin ou client MySQL. Ajoute la colonne <code>entity = 1</code> sur les 8 tables métier historiques (<code>llx_missionsplanet_mission</code> , <code>invoice_draft</code> , <code>invoice_draft_lines</code> , <code>tbl_client_billed</code> , <code>tbl_client_invoice_lines</code> , <code>tbl_credit_note_applications</code> , <code>tbl_mission_billed</code> , <code>tbl_invoice_sequence</code>). Idempotent : les erreurs "Duplicate column name" peuvent être ignorées.
3	Vérification post-migration	MySQL prod	Lancer le bloc <code>SELECT ... GROUP BY entity</code> commenté en fin de <code>requete_sql_entity1_backfill.sql</code> . Attendu : <code>entity = 1</code> partout, aucune ligne à <code>NULL</code> .
4	Déploiement du code	Front build + <code>api/</code> PHP	<code>npm run build</code> puis copie API + <code>composer install --no-dev</code> .
5	Migrations PHP automatiques	Fonctions <code>ensureXxxTable()</code>	Aucune action manuelle. Se déclenchent au premier appel HTTP de chaque endpoint concerné (<code>ensureInvoiceSequenceTable</code> , <code>ensureInvoiceDraftTable</code> , <code>ensureMissionEntityColumn</code> , <code>ensureInvoiceDraftLinesEntityColumn</code> , <code>ensureClientBillingTable</code> , <code>ensureCreditNoteApplicationsTable</code> , <code>ensureClientInvoiceLinesTable</code> , <code>ensureDupMapTable</code> , <code>ensureEntityBootstrapTable</code>). Complètent défensivement le script SQL et ajoutent les colonnes récentes (avoirs : <code>invoice_type</code> , <code>source_invoice_number</code> , <code>applied_amount</code> , <code>credit_consumed</code> , <code>credit_note_reason</code>).
6	Bootstrap 1 ^{er} login super-admin	<code>runEntityDataDuplicationOnce()</code>	Aucune action manuelle. Appelée par <code>api/login.php</code> . Duplique les référentiels entité 1 → entité 2. Verrouillée par <code>tbl_entity_bootstrap</code> — ne tourne qu'une seule fois.
7	(Optionnel) Audit d'intégrité	<code>api/dev/audit_invoice_integrity.php</code>	À lancer uniquement après incident (crash serveur, coupure). Détecte doublons de numéros, séquence désynchronisée, PDF orphelins. Non nécessaire lors d'un déploiement nominal.

Rappel de robustesse Le script SQL manuel (étape 2) et les `ensureXxxTable()` (étape 5) font

volontairement double emploi. Peu importe l'ordre : si l'un a déjà tourné, l'autre détecte que la colonne existe et passe. Objectif : aucun trou possible entre migration de données et déploiement du code.

À NE PAS lancer Les autres scripts de `api/dev/` ne sont **pas** des migrations :
`fix_promote_missions.php` (correction ponctuelle des statuts brouillon/validé),
`force_entity_duplication.php` (relance manuelle de la duplication référentiels — inutile si le 1^{er} login a bien tourné). À supprimer du serveur après déploiement.

7. Premier login post-déploiement — script de duplication

Le premier login super-admin déclenche automatiquement `runEntityDataDuplicationOnce()` (`api/entity_bootstrap.php`, appelé par `api/login.php`). Objectif : rendre l'entité 2 (PLANETE TRADUCTION FRANCE) autonome en clonant les **référentiels métier** de l'entité 1 (AMI historique) vers l'entité 2, sans jamais dupliquer deux fois.

Ce qui est dupliqué (dans cet ordre)

#	Table source	Contenu et particularité
1	<code>llx_societe</code> <i>Clients / tiers</i>	Toutes les colonnes copiées, <code>entity</code> forcé à 2. En cas de collision UNIQUE (<code>code_client</code> , <code>siren</code> , <code>siret</code> , <code>barcode</code>) → suffixe <code>-E1</code> , <code>-E2</code> , <code>-E3</code> .
2	<code>llx_socpeople</code> <i>Contacts</i>	Idem, avec <code>fk_soc</code> remappé vers le nouveau <code>rowid</code> du client cloné. Contact orphelin (parent non dupliqué) → ignoré, compté <code>contacts_orphaned</code> .
3	<code>llx_product</code> <i>Langues / produits / services</i>	<code>fk_product_parent</code> remappé sur le nouveau parent. Collision <code>ref/barcode</code> → suffixage.
4	<code>llx_c_payment_term</code> <i>Conditions de paiement</i>	UNIQUE (<code>entity</code> , <code>code</code>) déjà scopé par entité, pas de collision attendue.
5	<code>llx_user</code> <i>Utilisateurs / interprètes</i>	Login suffixé avec <code>_ptf</code> (<code>mdupont</code> → <code>mdupont_ptf</code>). Mot de passe inchangé (hash copié). <code>api_key</code> , <code>fk_socpeople</code> , <code>fk_member</code> remis à <code>NULL</code> (UNIQUE globales non scopées par entité).
5 bis	<code>tble_user_rights</code>	Droits utilisateurs recopiés via <code>INSERT IGNORE</code> .

Ce qui n'est PAS dupliqué (volontaire)

- **Comptes bancaires** (`tble_company_bank_accounts`) — l'admin PTF les crée à la main via Admin → Comptes bancaires.
- **Missions** (`llx_missionsplanet_mission`) — les missions historiques restent sur `entity=1`.
- **Factures / avoirs / brouillons** (`tble_client_billed`, `invoice_draft*`, `tble_credit_note_applications`) — données transactionnelles.
- **Séquences de numérotation** (`tble_invoice_sequence`) — la PK composite (`serie`, `entity`) fait démarrer l'entité 2 sur un compteur vierge.

Idempotence — mécanisme de sécurité

Chaque duplication est enregistrée dans `tbl_entity_dup_map` (`source_table`, `source_rowid`, `target_rowid`, `target_entity`). Avant chaque `INSERT`, `alreadyDuplicatedToEntity()` vérifie ce registre : si la ligne a déjà été clonée, on saute et on compte en `skipped`. **Relancer le script n'a aucun effet secondaire**, même 10 fois de suite.

Verrou temporel — 1 exécution par heure maximum

La table `tbl_entity_bootstrap` enregistre la date du dernier run par entité. Si un login intervient dans les 60 minutes qui suivent → sortie immédiate, aucune requête inutile. Passé ce délai, la fonction est rejouée pour **auto-heal** : si de nouvelles données ont été ajoutées côté `entity=1` entre-temps, elles seront clonées à leur tour vers `entity=2` (les autres, déjà mappées, sont ignorées).

Attendu — journal `api_error.log`

```
[entity_bootstrap] entity_bootstrap[2]: {
  "clients_source": 735, "clients_created": 735,
  "contacts_source": 1955, "contacts_created": 1953, "contacts_orphaned": 2,
  "products_source": 422, "products_created": 422,
  "payment_terms_source": 8, "payment_terms_created": 8,
  "users_source": 18, "users_created": 18,
  "rights_copied": 96
}
```

Premier login : 5-15 s selon la charge serveur. Logins suivants : instantanés.

Rattrapage manuel `api/dev/force_entity_duplication.php?token=AMI_FIX_2026_ENTITY_DUP` court-circuite le verrou 60 min et rejoue immédiatement la duplication. À utiliser uniquement si un ajout urgent sur `entity=1` doit être propagé sans attendre — sinon inutile. À supprimer du serveur après usage.

8. Checklist de vérification post-déploiement

#	Vérification	Comment tester
1	Login OK	Se connecter avec un compte <code>*_ptf</code> (<code>entity=2</code>). Un token JWT doit être délivré.
2	Liste des missions non vide	Page Missions → au moins une ligne visible. Filtrer par statut fonctionne.
3	Liste des tiers non vide	Page Tiers → SPADA / FORUM RÉFUGIÉS visibles. Cliquer sur "voir détails" → contacts affichés.
4	Cascade contact fonctionnelle	Nouvelle mission → sélectionner un tiers → la liste des personnes demandeuses se remplit.
5	Aperçu PDF facture	Facturation → bouton Aperçu PDF sur un brouillon → le PDF s'ouvre dans un nouvel onglet.

6	Émission facture	Émettre une facture définitive → numéro attribué de façon atomique, brouillon supprimé.
7	Création manuelle de devis	Devis → Nouveau devis → sélectionner client + lignes → PDF de devis généré.

9. Sécurité — points à vérifier avant mise en ligne

- Le dossier `api/dev/` contient des scripts de maintenance (`fix_promote_missions.php`, `force_entity_duplication.php`). À **supprimer** du serveur de prod ou à déplacer hors du webroot.
- Les logs d'erreur PHP sont écrits dans `api/api_error.log` — vérifier les permissions et exclure ce fichier de l'accès public via `.htaccess`.
- Sauvegarder la base de données **avant** le premier déploiement pour pouvoir rollback si nécessaire.
- Vérifier que `Content-Type: application/pdf` passe bien à travers d'éventuels proxys ou WAF (aperçu PDF).

10. Procédure de rollback

1. Restaurer `api/` et `index.html+assets/` depuis la sauvegarde précédente (idéalement un dossier daté par déploiement).
2. Si des tables custom ont été créées par erreur, elles sont idempotentes — laisser en place. Aucune donnée Dolibarr `llx_*` n'est modifiée par le déploiement.
3. Si la duplication de l'entité 2 a produit des doublons, restaurer un dump SQL antérieur au premier login. Toujours passer par un dump complet, pas des `DELETE FROM llx_societe WHERE entity=2` qui casseraient les FK.

11. Historique des versions

Version	Date	Modification
1.0	02/08/2026	Version initiale — déploiement React + PHP + mPDF, entité 2 (PTF).

Document généré automatiquement via `docs/generate_deployment_pdf.php`. Pour régénérer : `php docs/generate_deployment_pdf.php`.